Open in Colab

> **Run this in Google Colab.** Click the badge above (or open the notebook from Canvas), then run the **Setup** cell first — it installs the dependencies. No local install needed.

Assignment 3 · Milestone 3 — Model Adaptation Experiment

ISBA 2411 · Due Sunday, July 26, 2026 (11:59 PM PT) · Week 6 Team project: MedReview Insight — UCI Drug Review Dataset Team members: Varsha Pai, Zahra Fahimfar, Krystle Jozen Dario

This is the demo notebook's data swapped for our team's task: **binary sentiment classification of patient drug reviews** (`drugsComTrain_raw`), one of the components in our MedReview Insight pipeline (alongside ADE extraction and drug ranking). The metric is **accuracy** (plus macro-F1), exactly as in the demo.

Deliverables

- **Completed notebook** — this notebook, run on our data.
- **500-word analysis** — which adaptation strategy best fits our project, and why.

What to submit on Canvas

Submit **both** — the live work *and* an archival copy for the record:

1. **GitHub notebook link** — the URL to the team's completed notebook in the project repo (paste it in the *Website URL* box).
2. **PDF copy** — export the completed notebook to PDF (in Colab: *File → Print → Save as PDF*) and upload it (*File Upload*, PDF only).

> Canvas may only accept one submission method per attempt. Either way, **paste the GitHub notebook URL in the cell field below before**

> **exporting** so the link is also captured inside the PDF.

Team GitHub notebook URL: *(paste here before exporting to PDF)*

Reading connections

Strategy	Where to read
Prompt engineering (zero-/few-shot)	HOLLM Ch. 6
Fine-tuning representation models	HOLLM Ch. 11; Tunstall Ch. 2
Fine-tuning generative models / PEFT (LoRA)	HOLLM Ch. 12
Working with few labels	Tunstall Ch. 9

> Zero-shot, few-shot, and LoRA are the **default set**. Compare them on performance, cost, and effort.

> **How this notebook works.** Same pipeline as the demo, but Section 0 loads and labels `drugsComTrain_raw` instead of the toy product-review data — everything downstream (Strategies A/B/C, comparison table, analysis) runs off the same `train` / `test_texts` / `test_labels` variables unchanged.

> **Compute note.** Uses small models (`flan-t5-small`, `distilbert-base-uncased`) that run on Colab CPU; a GPU runtime is faster for the LoRA step. A fixed seed is set in setup. Drug reviews run much longer than the demo's product reviews, so prompts are truncated and the LoRA tokenizer uses a longer `max_length` — see Section 0.

How this is graded

Scored on the shared milestone rubric (`docs/ISBA2411_Assignment_Rubric.pdf`):

Rubric criterion	In this milestone
Execution	Three working adaptation strategies on the same task + metric
Analysis & Insight	A reasoned cost/quality/effort comparison and a recommendation
Communication	Clear analysis tied to the prompting vs. fine-tuning readings

```
# ===== Setup (Colab) =====
!pip install -q transformers datasets peft accelerate scikit-learn pa

import time, numpy as np, pandas as pd, torch, random
from sklearn.metrics import accuracy_score, f1_score
SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'
print('device:', DEVICE)
```

```
device: cuda
```

```
!
!pip uninstall -y torchao # uninstall the old version
!pip install torchao==0.16.0 # install a compatible version
```

```
Found existing installation: torchao 0.10.0
Uninstalling torchao-0.10.0:
  Successfully uninstalled torchao-0.10.0
Collecting torchao==0.16.0
  Downloading torchao-0.16.0-cp310-abi3-manylinux_2_24_x86_64.manylinu
Downloading torchao-0.16.0-cp310-abi3-manylinux_2_24_x86_64.manylinux_
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.2/3.2 MB 9.8 MB/s eta 0:
Installing collected packages: torchao
Successfully installed torchao-0.16.0
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import os
os.chdir('/content/drive/Shared drives/ISBA 2411 Group 1/Data/')
```

0. Task + data

Task: **binary sentiment** (positive / negative) on patient drug reviews from `drugsComTrain_raw`. Sentiment label is derived from the review's star `rating` (1–10): **rating ≥ 7 → positive**, **rating ≤ 4 → negative**; reviews in between (5–6, ambiguous/mixed) are dropped, same idea as filtering out neutral in a 3-class scheme. The metric is **accuracy** (plus macro-F1). Therefore, the labels represent patient ratings rather than manually annotated sentiment, which should be considered when interpreting the results.

We keep the exact same downstream contract as the demo: `train` and `test` lists of `(text, label)` pairs with integer labels (`0`/`1`), a `LABELS` name map, and `test_texts` / `test_labels`. Nothing in Strategies A–C below needed to change.

```
# ====== SWAP IN YOUR TEAM'S DATA HERE ======
import html

DATA_PATH = 'drugsComTrain_raw.xlsx'  # expected filename after uploa

# In Colab, upload the file if it isn't already sitting in the workin
# (skip this cell's upload prompt if you've mounted Drive and adjuste
import os
if not os.path.exists(DATA_PATH):
    try:
        from google.colab import files
        print(f'Please upload {DATA_PATH}')
        uploaded = files.upload()
        DATA_PATH = next(iter(uploaded))
    except ImportError:
        pass  # not running in Colab / file already present locally

raw = pd.read_excel(DATA_PATH)
raw = raw.dropna(subset=['review', 'rating']).copy()

def clean_review(t):
    t = str(t).strip()
    if t.startswith('"') and t.endswith('"'):
        t = t[1:-1]
    return html.unescape(t)
```

```
raw['review_clean'] = raw['review'].apply(clean_review)
raw['label'] = np.where(raw['rating'] >= 7, 1, np.where(raw['rating']
raw = raw.dropna(subset=['label'])
raw['label'] = raw['label'].astype(int)

# Reviews run much longer than the demo's product reviews (avg ~450 cl
# truncate for the prompting strategies so prompts stay within the mo
PROMPT_CHAR_LIMIT = 300
raw['review_prompt'] = raw['review_clean'].str.slice(0, PROMPT_CHAR_L

N_TRAIN_PER_CLASS = 100   # -> 200 train rows, balanced
N_TEST_PER_CLASS = 30     # -> 60 test rows, balanced

pos = raw[raw.label == 1].sample(N_TRAIN_PER_CLASS + N_TEST_PER_CLASS
neg = raw[raw.label == 0].sample(N_TRAIN_PER_CLASS + N_TEST_PER_CLASS

train_df = pd.concat([pos.iloc[:N_TRAIN_PER_CLASS], neg.iloc[:N_TRAIN_
test_df  = pd.concat([pos.iloc[N_TRAIN_PER_CLASS:], neg.iloc[N_TRAIN_

LABELS = {0: 'negative', 1: 'positive'}
train = list(zip(train_df['review_prompt'], train_df['label']))
test  = list(zip(test_df['review_prompt'],  test_df['label']))

test_texts  = [t for t, _ in test]
test_labels = [y for _, y in test]
print(f'train={len(train)}  test={len(test)}')
print('train label balance:', train_df['label'].value_counts().to_dic
print('test label balance:', test_df['label'].value_counts().to_dict(
```

```
train=200  test=60
train label balance: {1: 100, 0: 100}
test label balance: {1: 30, 0: 30}
```

1. Strategy A — Zero-shot prompting

Prompt an instruction-tuned model (`flan-t5-small`) with **no examples**.

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

flan_name = 'google/flan-t5-small'
flan_tok = AutoTokenizer.from_pretrained(flan_name)
flan = AutoModelForSeq2SeqLM.from_pretrained(flan_name).to(DEVICE)
```

```
def flan_generate(prompt, max_new_tokens=5):
    ids = flan_tok(prompt, return_tensors='pt', truncation=True, max_
    out = flan.generate(ids, max_new_tokens=max_new_tokens)
    return flan_tok.decode(out[0], skip_special_tokens=True)

def parse_sentiment(out):
    return 1 if 'positive' in out.lower() else 0

def zero_shot(text):
    prompt = (f'Is the sentiment of this patient drug review positive
              f'Review: "{text}"\nAnswer:')
    return parse_sentiment(flan_generate(prompt))

t0 = time.time()
pred_zs = [zero_shot(t) for t in test_texts]
zs = dict(strategy='zero-shot',
          accuracy=round(accuracy_score(test_labels, pred_zs), 3),
          macro_f1=round(f1_score(test_labels, pred_zs, average='macr
          sec=round(time.time()-t0, 1), trained_params=0)
print(zs)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenti
config.json: 100%                                   1.40k/1.40k [00:00<00:00, 131kB/s]
tokenizer_config.json: 100%                         2.54k/2.54k [00:00<00:00, 250kB/s]
spiece.model: reconstructing file: 100%               792kB /  792kB, 78.4kB/s
spiece.model: downloading bytes:                      649kB, 64.2kB/s
tokenizer.json: 100%                              2.42M/2.42M [00:00<00:00, 39.7MB/s]
special_tokens_map.json: 100%                       2.20k/2.20k [00:00<00:00, 225kB/s]
model.safetensors: reconstructing file: 100%          308MB /  308MB, 28.8MB/s
model.safetensors: downloading bytes:                 291MB, 25.9MB/s
Loading weights: 100%                             190/190 [00:00<00:00, 2916.29it/s]
[transformers] The tied weights mapping and config for this model speci
generation_config.json: 100%                        147/147 [00:00<00:00, 4.23kB/s]
{'strategy': 'zero-shot', 'accuracy': 0.717, 'macro_f1': 0.692, 'sec':
```

2. Strategy B — Few-shot prompting

Same model, but prepend a few labeled **in-context examples** to the prompt.

```
shots = train[:2] + train[-2:]  # 2 positive + 2 negative
preamble = 'Classify each patient drug review as positive or negative.
for t, y in shots:
    preamble += f'Review: "{t}"\nAnswer: {LABELS[y]}\n'

def few_shot(text):
    prompt = preamble + f'Review: "{text}"\nAnswer:'
    return parse_sentiment(flan_generate(prompt))

t0 = time.time()
pred_fs = [few_shot(t) for t in test_texts]
fs = dict(strategy='few-shot',
          accuracy=round(accuracy_score(test_labels, pred_fs), 3),
          macro_f1=round(f1_score(test_labels, pred_fs, average='macr
          sec=round(time.time()-t0, 1), trained_params=0)
print(fs)
```

```
{'strategy': 'few-shot', 'accuracy': 0.683, 'macro_f1': 0.648, 'sec':
```

3. Strategy C — LoRA fine-tuning

Fine-tune a small encoder (`distilbert-base-uncased`) with a **LoRA adapter** — only a tiny fraction of parameters are trained. Note: `DS` tokenizes from `review_clean` (untruncated to 300 chars) since the tokenizer does its own truncation at `max_length`, so fine-tuning can see more of each review than the prompting strategies did.

```
from transformers import (AutoTokenizer, AutoModelForSequenceClassifi
                          TrainingArguments, Trainer)
from peft import LoraConfig, get_peft_model, TaskType
import torch

ckpt = 'distilbert-base-uncased'
tok = AutoTokenizer.from_pretrained(ckpt)
```

```
MAX_LEN = 128  # longer than the demo's 64, since drug reviews run lo

# Rebuild train/test rows from the full (untruncated) cleaned review
train_full = list(zip(train_df['review_clean'], train_df['label']))
test_full  = list(zip(test_df['review_clean'],  test_df['label']))

class DS(torch.utils.data.Dataset):
    def __init__(self, rows):
        self.enc = tok([t for t, _ in rows], truncation=True, padding=
        self.y = [y for _, y in rows]
    def __len__(self): return len(self.y)
    def __getitem__(self, i):
        item = {k: torch.tensor(v[i]) for k, v in self.enc.items()}
        item['labels'] = torch.tensor(self.y[i]); return item

base = AutoModelForSequenceClassification.from_pretrained(ckpt, num_l
lora = LoraConfig(task_type=TaskType.SEQ_CLS, r=8, lora_alpha=16, lor
                  target_modules=['q_lin', 'v_lin'])
model = get_peft_model(base, lora)
trainable = sum(p.numel() for p in model.parameters() if p.requires_g
model.print_trainable_parameters()

args = TrainingArguments(output_dir='/tmp/lora_out', num_train_epochs=
                         per_device_train_batch_size=8, learning_rate=
                         logging_steps=50, report_to='none', seed=SEE
trainer = Trainer(model=model, args=args, train_dataset=DS(train_full
t0 = time.time(); trainer.train(); train_sec = round(time.time()-t0, 

import numpy as np
model.eval()
test_full_texts = [t for t, _ in test_full]
enc = tok(test_full_texts, truncation=True, padding=True, max_length=M
with torch.no_grad():
    pred_lora = model(**enc).logits.argmax(-1).cpu().tolist()
lora_res = dict(strategy='LoRA',
                accuracy=round(accuracy_score(test_labels, pred_lora),
                macro_f1=round(f1_score(test_labels, pred_lora, averag
                sec=train_sec, trained_params=trainable)
print(lora_res)
```

```
Loading weights: 100%                                   100/100 [00:00<00:00, 1050.07it/s]
[transformers] DistilBertForSequenceClassification LOAD REPORT from: di
Key                      | Status     |
-------------------------+------------+-
vocab_layer_norm.bias    | UNEXPECTED |
```

```
vocab_transform.weight   | UNEXPECTED |
vocab_projector.bias     | UNEXPECTED |
vocab_transform.bias     | UNEXPECTED |
vocab_layer_norm.weight  | UNEXPECTED |
pre_classifier.weight    | MISSING    |
pre_classifier.bias      | MISSING    |
classifier.bias          | MISSING    |
classifier.weight        | MISSING    |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/archite
- MISSING:      those params were newly initialized because missing fr
WARNING:torchao:Skipping import of cpp extensions due to incompatible t
trainable params: 739,586 || all params: 67,694,596 || trainable%: 1.09
[transformers] `use_return_dict` is deprecated! Use `return_dict` inste
```


[200/200 00:18, Epoch 8/8]

Step	Training Loss
50	0.632406
100	0.228834
150	0.015841
200	0.009039

```
{'strategy': 'LoRA', 'accuracy': 0.783, 'macro_f1': 0.782, 'sec': 20.9,
```

4. Compare

This table — built from the runs above — drives your analysis. `sec` is wall-clock cost (prompting = inference time; LoRA = training time); `trained_params` shows how little LoRA updates.

```
results = pd.DataFrame([zs, fs, lora_res])[['strategy','accuracy','ma
results
```

	strategy	accuracy	macro_f1	sec	trained_params
0	zero-shot	0.717	0.692	8.1	0
1	few-shot	0.683	0.648	7.1	0
2	LoRA	0.783	0.782	20.9	739586

5. Analysis (≈500 words)

Winner: LoRA fine-tuning. On our 60-review held-out test set, LoRA reached 0.783 accuracy / 0.782 macro-F1, beating zero-shot (0.717 / 0.692) and few-shot (0.683 / 0.648). LoRA updates only 1.09% of DistilBERT's parameters (739K of 67.7M), providing an efficient fine-tuning approach while maintaining low additional inference cost after training. LoRA required an additional 20.9 seconds of training, whereas the prompting strategies required no training and only incurred inference time; because these measurements represent different stages of the workflow, they should not be interpreted as directly comparable runtimes. Sentiment classification is one input among several in our pipeline (alongside ADE extraction and drug ranking), and those downstream steps need a stable, predictable interface: a fine-tuned model gives us consistent latency and output format at inference time, where prompting behavior can vary with prompt wording or future model updates, making results less consistent across different prompt formulations. At the scale of the full drugsComTrain_raw set (~160K reviews), that consistency and per-inference cost matter far more than they do on a 60-row test.

A result worth flagging: few-shot actually *underperformed* zero-shot (0.683 vs 0.717). We didn't expect this. One possible explanation is that adding four in-context examples increased prompt complexity and introduced distracting context for the relatively small FLAN-T5-Small model, reducing its ability to focus on the target review. This is a concrete illustration of HOLLM Ch. 6's point that prompting quality is fragile to exact wording and example choice — more examples isn't automatically better, especially on smaller models.

When prompting still wins. Zero/few-shot needed no labeled training data and no training infrastructure — we had predictions within seconds of writing the prompt. That's the right call early in a project, before we've committed to a label scheme, or if MedReview Insight needs to classify an entirely new category (e.g., side-effect severity) with only a handful of examples on hand. It's also the more honest choice if reviewer volume stays low — the fixed cost of setting up a LoRA training pipeline isn't worth it for occasional, small-batch use.

When fine-tuning wins. HOLLM Ch. 11–12 frame full fine-tuning and PEFT methods like LoRA as trading upfront training cost for efficient, specialized inference — exactly what we saw. Unlike the prompting strategies, which truncated each review to approximately 300 characters before prompting, the LoRA model tokenized the cleaned review directly and truncated only at the tokenizer's maximum sequence length (128 tokens). This provides a more consistent preprocessing pipeline while allowing the tokenizer to determine the optimal token boundaries, so the model saw more of each review's context. For a component that will eventually run against the full ~160K-review dataset, that combination of higher accuracy, longer context, and efficient marginal inference cost makes fine-tuning the clear long-term choice, even though it's the only strategy that required real training and a GPU runtime.

Data. Our labels are a proxy — rating ≥7 mapped to positive, ≤4 to negative — not human-annotated sentiment, so some of our "errors" may really be label noise rather than model error. Tunstall Ch. 9 argues that even small amounts of high-quality labeled data disproportionately help fine-tuning methods; moving from 200 training rows to a few thousand, and from rating-threshold labels to human-reviewed sentiment tags, would likely widen LoRA's lead further, since it's the only strategy that can directly learn from more/better labels rather than just fitting more examples into a limited prompt context window.

Although LoRA achieved the highest performance in this experiment, the improvement over the zero-shot approach was relatively modest (approximately 6.6 percentage points in macro F1). This suggests that prompt-based methods remain practical when labeled training data or computational resources are limited, while LoRA provides the best balance between accuracy and parameter efficiency when task-specific fine-tuning is feasible.

Conclusion. Among the three adaptation strategies, LoRA achieved the best overall

performance, producing the highest accuracy and macro F1 score while updating only 1.09% of the model's parameters. Although LoRA required additional training time, inference remained efficient after fine-tuning. Zero-shot prompting provided a strong baseline without requiring any training, whereas few-shot prompting did not improve performance, likely because the additional examples increased prompt complexity for the relatively small FLAN-T5-Small model. Overall, the results demonstrate that parameter-efficient fine-tuning offers the best balance between accuracy, computational efficiency, and scalability for this sentiment classification task, while prompt-based approaches remain valuable when labeled training data or fine-tuning resources are unavailable.